Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 3
з дисципліни «Професійний Python»
на тему: «Ітератори, генератори та потокова обробка даних»

Виконав:
студент 3 курсу
групи ФеП-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, ключовими словами yield та yield from, генераторними виразами (generator expressions), модулем itertools, концепцією лінивих обчислень (lazy evaluation) та потоковою обробкою великих наборів даних. Навчитися будувати багатоетапні потокові конвеєри (streaming pipelines) без повної матеріалізації колекцій у пам'яті, обмежувати потік через islice, реалізовувати пакетну обробку (batching) та експериментально оцінювати використання оперативної пам'яті (tracemalloc) і часову ефективність порівняно з жадібним (eager) підходом.

2. Номер індивідуального варіанта

Варіант 15. Аналіз системних логів

Постановка задачі:

Створити модуль потокової обробки логів програмної системи для великого набору даних (від 100 000+ записів).

Формат вхідного запису (CSV):

timestamp, level, module, request_id, message

Вимоги до реалізації:

Потокове читання лог-файлу (streaming log reader) без завантаження всього вмісту в RAM через yield from.

Потоковий синтаксичний аналіз (parsing) та валідація рядків.

Фільтрація подій рівня ERROR.

Фільтрація логів за назвою підсистеми (module).

Підрахунок розподілу подій за рівнями логування (count log levels) без створення проміжного списку.

Отримання перших N помилок через itertools.islice.

Пакетна обробка записів (batching).

Групування послідовних рівнів через itertools.groupby.

Потоковий пошук запису за request_id із достроковим завершенням (early termination).

Підсумковий аналітичний звіт (summary) без збереження повного лог-файлу в оперативній пам'яті.

Власний клас-ітератор із реалізацією __iter__ та __next__.

Експериментальне порівняння eager та lazy версій за часом виконання та піковою пам'яттю (tracemalloc).

3. Схема конвеєра (Streaming Pipeline Architecture)

```
None
[ data/system_logs.csv ] (150 000+ рядків)
           |
           ▼ (yield from file)
      read_lines()             <- Потокове зчитування
рядків із диска
           |
           ▼ (yield LogRecord)
     parse_csv_rows()          <- Парсинг CSV та валідація
полів
           |
           ▼ (yield LogRecord)
 normalize_log_messages()      <- Потокова нормалізація
тексту повідомлень
           |
           ▼ (yield LogRecord)
  filter_by_level / module     <- Лінива фільтрація (ERROR
/ назва модуля)
           |

┌──────────┴──────────────────────────────┴──────────────────────┐
  ▼                                        ▼
▼
islice()                       batched_logs()
calculate_log_summary()
(Перші N записів)              (Пакети фікс. розм.)
(Потокова агрегація)
```

4. Структура проєкту

```
None
Lab_03/
```

```
├── .gitignore
├── pyproject.toml
├── README.md
├── generate_data.py
├── main.py
├── data/
│   └── system_logs.csv
├── report/
│   └── Lab_03_Report.pdf
└── src/
    └── log_processor/
        ├── __init__.py
        ├── models.py
        ├── readers.py
        ├── parsers.py
        ├── filters.py
        ├── transformations.py
        ├── batches.py
        ├── analytics.py
        └── pipeline.py
```

5. Опис створених модулів

generate_data.py: утиліта генерації синтетичного датасету системних логів обсягом 150 000 рядків із псевдовипадковими мітками часу, рівнями (INFO, DEBUG, WARNING, ERROR, CRITICAL), модулями, request_id та повідомленнями.

src/log_processor/__init__.py: ініціалізатор пакета, що забезпечує коректну роботу в режимі розробника (pip install -e .).

src/log_processor/models.py: містить модель даних LogRecord (на базі NamedTuple), а також власний клас-ітератор SimpleLogRangeIterator, який реалізує протокол ітератора (__iter__, __next__, StopIteration).

src/log_processor/readers.py: модуль потокового зчитування сирих рядків із файлового дескриптора за допомогою конструкції yield from.

src/log_processor/parsers.py: синтаксичний аналізатор рядків на базі csv.DictReader, що валідує наявність усіх полів та ліниво генерує екземпляри LogRecord.

src/log_processor/filters.py: містить функції-генератори для фільтрації за рівнем логування (filter_by_level), модулем (filter_by_module), а також функцію лінійного потокового пошуку find_by_request_id із достроковим виходом (early termination).

src/log_processor/transformations.py: генератор нормалізації повідомлень (форматування регістру без мутації незмінного кортежу через _replace).

src/log_processor/batches.py: реалізація пакетної обробки потоку batched_logs на базі itertools.islice.

src/log_processor/analytics.py: функції потокової агрегації без матеріалізації списку (calculate_log_summary на базі Counter) та групування послідовних рівнів через itertools.groupby.

src/log_processor/pipeline.py: фабрика збирання потокового конвеєра build_log_pipeline, що об'єднує читанку, парсинг, нормалізацію та фільтрацію в єдиний ітератор.

main.py: головна точка входу, яка демонструє виконання завдань варіанта 15, роботу власного ітератора, функцій itertools, а також порівнює ресурсоємність жадібної (eager) та лінивої (lazy) реалізацій через tracemalloc.

6. Текст основних фрагментів коду

src/log_processor/models.py

```python
from collections.abc import Iterator
from typing import NamedTuple

class LogRecord(NamedTuple):
    timestamp: str
    level: str
    module: str
    request_id: str
    message: str

class SimpleLogRangeIterator:
    """Власний клас-ітератор для демонстрації Iterator
Protocol."""
```

```
    def __init__(self, records: list[LogRecord], stop: int)
-> None:
        self.records = records
        self.stop = min(stop, len(records))
        self.current = 0

    def __iter__(self) -> Iterator[LogRecord]:
        return self

    def __next__(self) -> LogRecord:
        if self.current >= self.stop:
            raise StopIteration
        record = self.records[self.current]
        self.current += 1
        return record
```

src/log_processor/readers.py

```python
from collections.abc import Iterator
from pathlib import Path

def read_lines(path: Path) -> Iterator[str]:
    """Потокове читання файлу без завантаження всього
вмісту в RAM."""
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

src/log_processor/parsers.py

```python
import csv
from collections.abc import Iterable, Iterator
from log_processor.models import LogRecord
```

```
def parse_csv_rows(lines: Iterable[str]) ->
Iterator[LogRecord]:
    """Генератор парсингу та базової валідації записів
логів."""
    reader = csv.DictReader(lines)
    for row in reader:
        try:
            timestamp = row["timestamp"].strip()
            level = row["level"].strip()
            module = row["module"].strip()
            request_id = row["request_id"].strip()
            message = row["message"].strip()

            if not (timestamp and level and module and
request_id and message):
                continue

            yield LogRecord(
                timestamp=timestamp,
                level=level,
                module=module,
                request_id=request_id,
                message=message,
            )
        except (KeyError, TypeError, ValueError):
            continue
```

src/log_processor/filters.py

Python

```
from collections.abc import Iterable, Iterator
from log_processor.models import LogRecord

def filter_by_level(records: Iterable[LogRecord],
target_level: str) -> Iterator[LogRecord]:
    """Фільтрація логів за рівнем (наприклад, ERROR)."""
```

```
    for record in records:
        if record.level == target_level:
            yield record

def filter_by_module(records: Iterable[LogRecord],
target_module: str) -> Iterator[LogRecord]:
    """Фільтрація логів за назвою модуля."""
    for record in records:
        if record.module == target_module:
            yield record

def find_by_request_id(records: Iterable[LogRecord],
target_id: str) -> LogRecord | None:
    """Потоковий пошук запису за request_id з early
termination."""
    for record in records:
        if record.request_id == target_id:
            return record
    return None
```

src/log_processor/batches.py

```python
from collections.abc import Iterable, Iterator
from itertools import islice
from log_processor.models import LogRecord

def batched_logs(records: Iterable[LogRecord], batch_size:
int) -> Iterator[list[LogRecord]]:
    """Розбиття потоку логів на пакети заданого розміру."""
    iterator = iter(records)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch
```

src/log_processor/analytics.py

```python
from collections import Counter
from collections.abc import Iterable, Iterator
from itertools import groupby
from log_processor.models import LogRecord

def calculate_log_summary(records: Iterable[LogRecord]) ->
dict:
    """Потокова агрегація статистики без повної
матеріалізації списку в пам'яті."""
    total_count = 0
    level_counts = Counter()
    module_counts = Counter()

    for record in records:
        total_count += 1
        level_counts[record.level] += 1
        module_counts[record.module] += 1

    return {
        "total_records": total_count,
        "level_counts": dict(level_counts),
        "module_counts": dict(module_counts),
    }

def group_consecutive_levels(records: Iterable[LogRecord])
-> Iterator[tuple[str, int]]:
    """Групування послідовних однакових рівнів логів через
itertools.groupby."""
    for level, group in groupby(records, key=lambda item:
item.level):
        count = sum(1 for _ in group)
        yield level, count
```

7. Пояснення pyproject.toml

Конфігураційний файл стандарту PEP 518/621:

[build-system]: визначає setuptools>=61.0 і setuptools.build_meta як бекенд побудови пакета.

[project]: декларує метадані застосунку (name = "log-processor", версію 0.1.0, вимоги до Python requires-python = ">=3.10").

[tool.setuptools.packages.find]: налаштовує автоматичне виявлення коду в папці src, що дозволяє встановлювати пакет у режимі редагування (python -m pip install -e .) для використання чистих абсолютних імпортів.

8. Вміст .gitignore

Файл запобігає потраплянню службових файлів інтерпретатора, артефактів збирання та важких тестових файлів у систему контролю версій Git:

```
None
__pycache__/
*.py[cod]
*$py.class
*.egg-info/
dist/
build/
.venv/
env/
data/*.csv
```

9. Результати виконання програми

Консольне виведення main.py:

```
None
=== ДЕМОНСТРАЦІЯ ЗАВДАНЬ ВАРІАНТА 15 ===

1. Перші 5 ERROR-записів (через islice):
  [2026-01-01T10:00:04] payment (req_7967): Slow query
detected
  [2026-01-01T10:00:06] worker (req_9080): Connection
refused to host
  [2026-01-01T10:00:16] payment (req_2601): Slow query
detected
```

```
  [2026-01-01T10:00:38] api (req_7658): Timeout while
waiting for response
  [2026-01-01T10:00:39] database (req_6121): Timeout while
waiting for response

2. Підрахунок рівнів логів (Streaming Summary):
  CRITICAL  : 30156
  DEBUG     : 30156
  ERROR     : 30070
  INFO      : 29759
  WARNING   : 29859

3. Перший пакет (batch) для модуля 'auth' (розмір 3):
  DEBUG -> Connection established
  DEBUG -> Timeout while waiting for response
  DEBUG -> Timeout while waiting for response

4. Групування послідовних рівнів (перші 15 записів через
groupby):
  Послідовність рівнів CRITICAL: 2 шт.
  Послідовність рівнів DEBUG: 1 шт.
  Послідовність рівнів ERROR: 1 шт.
  Послідовність рівнів DEBUG: 1 шт.
  Послідовність рівнів ERROR: 1 шт.
  Послідовність рівнів INFO: 2 шт.
  Послідовність рівнів WARNING: 1 шт.
  Послідовність рівнів CRITICAL: 1 шт.
  Послідовність рівнів WARNING: 1 шт.
  Послідовність рівнів INFO: 2 шт.
  Послідовність рівнів WARNING: 1 шт.
  Послідовність рівнів CRITICAL: 1 шт.

5. Пошук за request_id 'req_7967':
  Знайдено: LogRecord(timestamp='2026-01-01T10:00:04',
level='ERROR', module='payment', request_id='req_7967',
message='Slow query detected')
```

```
6. Generator expression (перше критичне повідомлення):
Connection established

=== ПОРІВНЯННЯ EAGER vs LAZY ===
Eager: Час = 3.160 с | Пікова пам'ять = 89.96 MB
Lazy:  Час = 3.479 с | Пікова пам'ять = 0.20 MB
Економія пам'яті: у 441.6 разів менше використання RAM у
Lazy версії.

7. Власний iterator (SimpleLogRangeIterator):
  [Власний ітератор] ERROR: Slow query detected
  [Власний ітератор] ERROR: Connection refused to host

8. itertools.takewhile (читати логи до першого не-ERROR):
  Знайдено підряд на початку файлу: 0 шт.
```

10. Порівняльний аналіз Eager vs Lazy Evaluation

Підхід	**Час виконання (с)**	**Пікове використання RAM (tracemalloc)**	**Економія пам'яті**
Eager (матеріалізація list[dict])	**3.160 с**	**89.96 MB**	Базовий рівень
Lazy (потоковий конвеєр через генератори)	**3.479 с**	**0.20 MB**	**у 441.6 разів менше**

Аналіз результатів:

Використання пам'яті: Лінива модель (Lazy Pipeline) показала суттєву перевагу, знизивши пікове навантаження на RAM з **89.96 MB** до **0.20 MB** (економія у ~441 раз). У лінивому підході пам'ять виділяється виключно під поточний оброблюваний екземпляр LogRecord, тоді як Eager-підхід одночасно завантажує весь масив зі 150 000 об'єктів у RAM.

Час виконання: Eager-підхід виявився дещо швидшим (3.160 с проти 3.479 с). Це пояснюється додатковими накладними витратами середовища виконання Python на збереження і відновлення контексту функцій-генераторів (frame switching) при кожному виклику yield.

Early Termination та Time to First Result: Lazy-підхід дозволяє отримати перший результат (islice або пошук за request_id) миттєво, обробивши лише перші кілька рядків і звільнивши ресурси без потреби повного обходу файлу.

11. Відповіді на контрольні запитання

Що таке iterable?
Об'єкт, що реалізує метод __iter__(), який повертає ітератор (списки, кортежі, рядки, файли).

Що таке iterator?
Об'єкт, який реалізує протокол ітератора: методи __iter__() та __next__(), зберігає поточний стан обходу і піднімає виняток StopIteration після завершення послідовності.

Чим iterable відрізняється від iterator?
Iterable — це джерело даних/контейнер, з якого можна отримати ітератор; iterator — це механізм безпосереднього обходу, який пам'ятає поточну позицію і є одноразовим.

Для чого використовується iter()?
Для отримання об'єкта ітератора з ітерабельного об'єкта.

Для чого використовується next()?
Для вилучення наступного елемента з ітератора.

Що робить __iter__()?
Повертає сам об'єкт ітератора.

Що робить __next__()?
Повертає наступне значення або викликає виняток StopIteration.

Що таке StopIteration?
Вбудований виняток Python, що сигналізує про вичерпання елементів в ітераторі.

Як цикл for використовує iterator protocol?
Викликає iter() на об'єкті, у нескінченному циклі викликає next() і коректно перехоплює StopIteration, перериваючи цикл.

Чому iterator часто називають one-shot object?
Тому що після повного проходження по ньому повторна ітерація неможлива (він залишається вичерпаним).

Чому iter(iterator) is iterator?
За протоколом метод __iter__() ітератора завжди повертає посилання на самого себе (self).

Чи можна повторно використати exhausted generator?
Ні, вичерпаний генератор завжди повертатиме StopIteration; його потрібно створювати заново.

Що таке generator function?
Функція, що містить хоча б одне ключове слово yield і повертає об'єкт-генератор замість прямого значення.

Яку роль виконує yield?
Повертає проміжне значення викликаючій стороні, призупиняє виконання функції та зберігає весь її локальний стан.

Чим yield відрізняється від return?
return остаточно завершує виконання функції та знищує її стек-фрейм; yield тимчасово заморожує виконання.

Коли фактично починає виконуватися generator function?
Лише тоді, коли на створеному генераторі вперше викликається функція next() (або починається цикл for).

Що зберігає generator між викликами next()?
Локальні змінні, стек виклику, стан циклів і поточний показник інструкції байт-коду.

Що таке generator expression?
Компактний лінивий синтаксис створення генератора у круглих дужках: (x for x in iterable).

Чим generator expression відрізняється від list comprehension?
List comprehension відразу виділяє пам'ять під усі елементи; generator expression повертає генератор і вираховує значення по одному за запитом.

Що таке lazy evaluation?
Стратегія обчислень, за якої значення обчислюються тільки в момент їх фактичної потреби.

Що таке eager evaluation?
Стратегія, коли всі вирази обчислюються та матеріалізуються в пам'яті негайно.

Які переваги lazy evaluation?
Мінімальне пікове споживання пам'яті, можливість роботи з нескінченними потоками, миттєвий time-to-first-result, ефективний early termination.

Які недоліки lazy evaluation?
Неможливість індексації за , одноразовість використання, незначні накладні витрати часу на перемикання стек-фреймів.

Що таке streaming?
Потокова обробка даних порціями або по одній одиниці без попереднього завантаження всього набору в пам'ять.

Чому streaming важливий для великих files?
Дозволяє опрацьовувати файли розміром у десятки гігабайт на машинах із невеликим обсягом оперативної пам'яті без ризику виникнення помилки MemoryError.

Чому read() може бути проблемою для multi-GB file?
Метод завантажує весь вміст файлу у вигляді одного гігантського рядка, що призводить до вичерпання RAM або аварійного завершення процесу (OOM).

Чому file object є iterable?
У класі файлового дескриптора реалізовано метод __iter__(), що повертає рядки файлу за допомогою буферизованого потокового читання.

Для чого використовується yield from?
Для прозорої делегації генерації значень іншому генератору чи ітерабельному об'єкту без використання явного циклу for.

Що таке itertools?
Стандартний модуль Python, що містить високоефективні швидкісні функції для побудови складних ітераторів на мові C.

Для чого itertools.count()?
Створює нескінченний числовий лінивий ітератор із заданим початком і кроком.

Чому count() потрібно обмежувати?
Оскільки він нескінченний, спроба матеріалізувати його через list(count()) спричинить нескінченний цикл і вичерпання пам'яті.

Для чого islice()?
Потоковий зріз ітератора: дозволяє вилучити діапазон елементів (наприклад, перші) без генерації решти послідовності.

Для чого chain()?
Послідовно об'єднує декілька ітерабельних об'єктів в один неперервний потік.

Як працює takewhile()?
Повертає елементи з потоку доти, доки умова є істинною; при першому ж значенні False повністю припиняє ітерацію.

Як працює dropwhile()?
Пропускає початкові елементи, поки умова True, і починає видавати всі наступні елементи, щойно умова вперше стала False.

Для чого accumulate()?
Обчислює накопичувальні суми або результати застосування довільної бінарної функції над послідовністю.

Для чого pairwise()?
Повертає послідовні пари сусідніх елементів потоку, що перекриваються ((s0, s1), (s1, s2), ...).

Як працює groupby()?
Групує підряд розташовані однакові елементи або елементи зі спільним результатом функції-ключа.

Чому перед groupby() часто потрібне sorting?
Тому що groupby не сканує всю колекцію, а формує нову групу щоразу, коли змінюється значення ключа поточного елемента.

Що робить product()?
Обчислює декартовий добуток вхідних послідовностей (аналог вкладених циклів for).

Чим combinations() відрізняється від permutations()?
У комбінаціях (combinations) порядок елементів не має значення і перестановки не враховуються; у перестановках (permutations) враховуються всі можливі варіанти розташування.

Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?
Через комбінаторний вибух: загальна кількість можливих станів може сягати трильйонів, що призведе до нескінченного часу виконання.

Що таке batching?
Техніка групування потокових елементів у невеликі пачки фіксованого розміру (наприклад, по 1000 елементів).

Які переваги batching?
Зменшення кількості мережевих запитів або операцій дискового вводу-виводу при збереженні контрольованого споживання пам'яті.

Що таке lazy pipeline?
Архітектурний патерн, у якому обробка даних організована у вигляді ланцюжка генераторів, де кожен наступний етап запитує дані в попереднього тільки за потреби.

Які stages може містити pipeline?
Source (джерело), parsing (парсинг), validation (валідація), filtering (фільтрація), transformation (перетворення), aggregation / sink (споживання/збереження).

Що є source у streaming pipeline?
Початкова точка потоку: файл, сокет, черга повідомлень або системний генератор.

Що є transformation?
Етап конвеєра, що модифікує, очищує або збагачує окремі записи на льоту.

Що є sink?
Кінцевий споживач потоку: функція агрегації, запис у БД, консольне виведення або експорт у новий файл.

Чи гарантує generator менший total execution time?
Ні, за рахунок накладних витрат на перемикання стек-фреймів генератор може працювати трохи повільніше за жадібний код на мові C.

Яка основна memory advantage generator?
Константне або мінімальне пікове використання оперативної пам'яті , що не залежить від загального розміру датасету.

Що таке time to first result?
Час від запуску програми до отримання найпершого обробленого елемента.

Чому lazy pipeline добре підтримує early termination?
Тому що обробка зупиняється в момент виконання умови (наприклад, знаходження елемента), не витрачаючи час на читання решти файлу.

Чому exception у generator може виникати із затримкою?
Тіло генератора не виконується в момент його виклику; виняток виникає лише тоді, коли споживач дійде до проблемного рядка під час виклику next().

Чому list(generator) може бути проблемою?
Виклик примусово завантажує абсолютно весь обсяг даних у пам'ять, нівелюючи перевагу генератора і ризикуючи викликати MemoryError.

Що трапиться при list(count())?
Програма зависне в нескінченному циклі накопичення чисел, поки операційна система не завершить процес через вичерпання RAM.

Як generator може впливати на lifecycle file resource?
Якщо генератор утримує контекстний менеджер with open(...), дескриптор файлу залишається відкритим доти, доки генератор не буде повністю вичерпано або знищено збирачем сміття (GC).

Як тестувати невеликий generator?
Шляхом його повної матеріалізації в тестовому оточенні: assert list(my_gen) == expected.

Як тестувати infinite generator?
Шляхом обмеження потоку за допомогою itertools.islice(my_gen, N) з наступною перевіркою зрізу.

Коли краще використовувати list, а коли generator?
List обирають, коли потрібен багаторазовий прохід, пряма індексація за чи сортування на місці, а набір даних гарантовано вміщується в RAM. Generator незамінний для великих файлів, потоків та послідовностей невідомої довжини, де критична економія оперативної пам'яті.

12. Висновок

У ході виконання лабораторної роботи № 3 було опановано ітераційну модель Python, протокол ітератора, функції-генератори та ліниву обробку даних. Для 15-го варіанта («Аналіз системних логів») розроблено повноцінний потоковий конвеєр, що включає streaming reader (yield from), парсер CSV, валідацію, фільтри подій ERROR та підсистем, нормалізацію, пакетну обробку (batched), групування послідовностей (groupby) та пошук за request_id з раннім перериванням.

Експериментальне профілювання пам'яті за допомогою tracemalloc на наборі зі 150 000 записів підтвердило ключову теоретичну перевагу генераторів: Lazy-підхід спожив лише **0.20 MB** пікової пам'яті проти **89.96 MB** у Eager-підході (економія у **441.6 разів**), забезпечивши стабільність роботи без загрози переповнення пам'яті.